INTRODUCTION

1.1. Problem Definition

Beverages consumed in daily life contain various chemical compounds that can affect human health, either positively or negatively. Traditional methods for analyzing beverage quality are often time-consuming, subjective, and require complex laboratory equipment. There is a need for a reliable, real-time, and cost-effective system that can accurately assess the flavor profile and chemical safety of beverages. This project addresses this need by developing an Electronic Tongue (E-Tongue) system that uses sensor data and machine learning techniques to evaluate beverage quality and determine whether a beverage is safe for consumption.

1.2. Overview of Project

This system will have the below functions / features as its detailed requirements

.

- **To develop an E-Tongue system** that mimics human taste perception using sensors such as pH, conductivity, gas, and temperature sensors.
- **To collect sensor-based data** from different beverage samples for feature extraction related to taste and chemical properties.
- **To analyze the sensor data** to determine taste attributes such as sweetness, bitterness, and acidity.
- **To train a machine learning model (Decision Tree)** using the collected data to classify beverages as consumable or non-consumable.
- **To evaluate the performance and accuracy** of the developed model in predicting beverage quality and safety.
- **To provide a real-time and automated solution** for beverage quality analysis, aiding food safety and quality control processes.

.

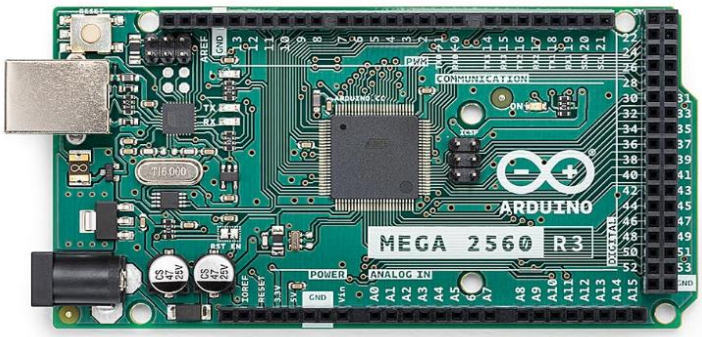
SYSTEM CONFIGURATION

2.1. HARDWARE SPECIFICATION:

- **Hardware Requirements System** :Windows XP
- **Microcontroller** :Raspberry pi 4, Arduino UNO
- **Sensors** :PH Sensor, Conductivity Sensor, gas, DHT11 Sensor
- **Display** :OLED

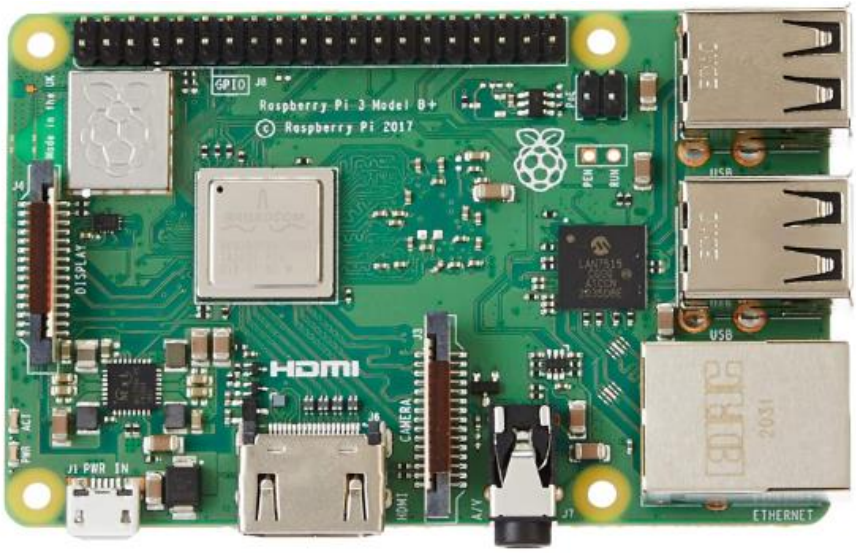
ABOUT THE HARDWARE:

ARDUINO:



Arduino is an open-source electronics platform based on easy-to-use hardware and software. It consists of a microcontroller (a small computer on a single chip) and an integrated development environment (IDE) that allows users to write, compile, and upload code to the board. Arduino is popular among hobbyists, engineers, and makers for creating interactive projects such as robotics, home automation, and IoT devices. It simplifies the process of controlling sensors, motors, lights, and other electronic components through programming. The platform supports various boards, such as Arduino Uno, Nano, and Mega, and has a large online community offering resources, tutorials, and libraries.

RASPBERRY PI:



Raspberry Pi is a small, affordable, single-board computer developed by the Raspberry Pi Foundation. It was created to promote computer science education and facilitate digital skills development, especially in developing countries. The Raspberry Pi is widely used for a variety of applications such as DIY projects, learning programming, robotics, home automation, and even as a low-cost server.

It features an ARM-based CPU, RAM, GPIO pins for hardware interfacing, and connectivity options like HDMI, USB, and Wi-Fi (in newer models). Raspberry Pi runs on various operating systems, with Raspberry Pi OS (formerly Raspbian) being the most common. Its versatility and low cost have made it popular among hobbyists, educators, and even professionals for prototyping and creating innovative tech solutions.

GLASS PH ELECTRODE:

A glass pH electrode is a type of sensor used to measure the acidity or alkalinity (pH) of a solution. It consists of a glass membrane that is sensitive to hydrogen ions (H^+), which are responsible for the pH of a solution. The electrode typically has two main parts:

1. **The Glass Membrane**: This is usually made from a special glass that allows hydrogen ions to pass through, generating a voltage that is proportional to the pH of the solution.

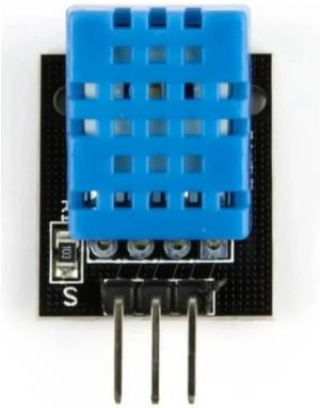
2. **The Reference Electrode**: This is placed in a solution with a stable, known potential (usually a potassium chloride solution) and provides a reference voltage for measuring the pH.

CONDUCTIVITY SENSOR:

A conductivity sensor is an instrument used to measure the electrical conductivity (EC) of a solution, which indicates the ability of the solution to conduct an electric current. This is primarily influenced by the presence and concentration of dissolved ions, such as salts, acids, and bases, in the solution.

The basic principle behind a conductivity sensor is that when an electric potential is applied between two electrodes, ions in the solution carry the current. The amount of current that flows between the electrodes is directly proportional to the ion concentration in the solution. The sensor measures this current flow and calculates the conductivity, usually expressed in siemens per centimeter (S/cm) or microsiemens per centimeter (µS/cm).

DHT11 SENSOR:



The DHT11 sensor is a low-cost, digital device used to measure temperature and humidity in a variety of applications. It offers a temperature range of 0°C to 50°C with an accuracy of ±2°C and a humidity range of 20% to 80% relative humidity with an accuracy of ±5%. Its digital output simplifies the process of interfacing with microcontrollers like Arduino or Raspberry Pi, using a single-wire communication protocol. While the DHT11 is popular for DIY projects, home automation, and basic weather stations, it has some limitations, including lower accuracy and a narrower measurement range compared to more advanced sensors. Despite these limitations, its affordability and ease of use make it an excellent choice for general indoor environmental monitoring and beginner-level electronics projects.

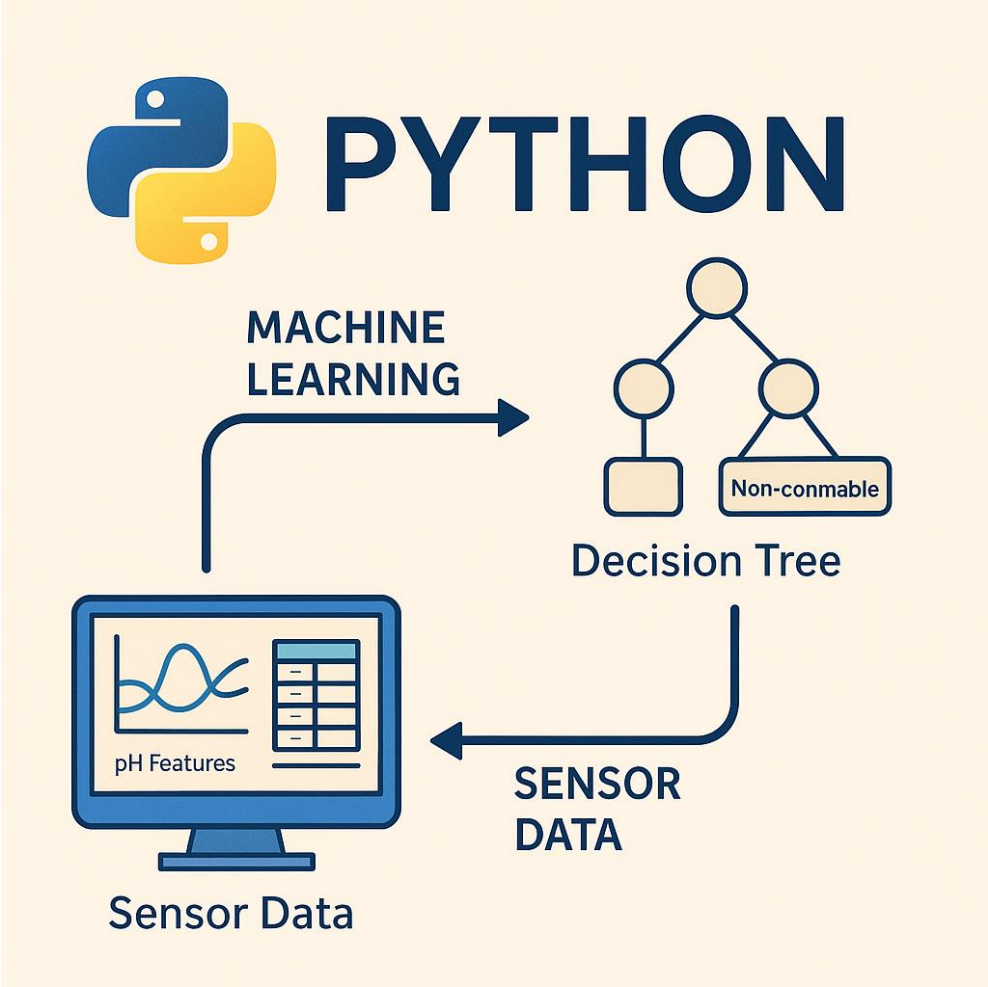
GAS SENSOR:

A gas sensor is a device used to detect the presence and concentration of gases in the environment. These sensors are commonly used in safety systems, industrial applications, and environmental monitoring to detect harmful or combustible gases, ensuring safety and proper air quality. Gas sensors typically work by detecting changes in the electrical properties, resistance, or chemical reactions caused by the target gas.

2.2. SOFTWARE SPECIFICATION:

- Language : Python
- Online dashboard : Thingspeak
- Tool : Google colab
- IDE : Thonny

PYTHON:



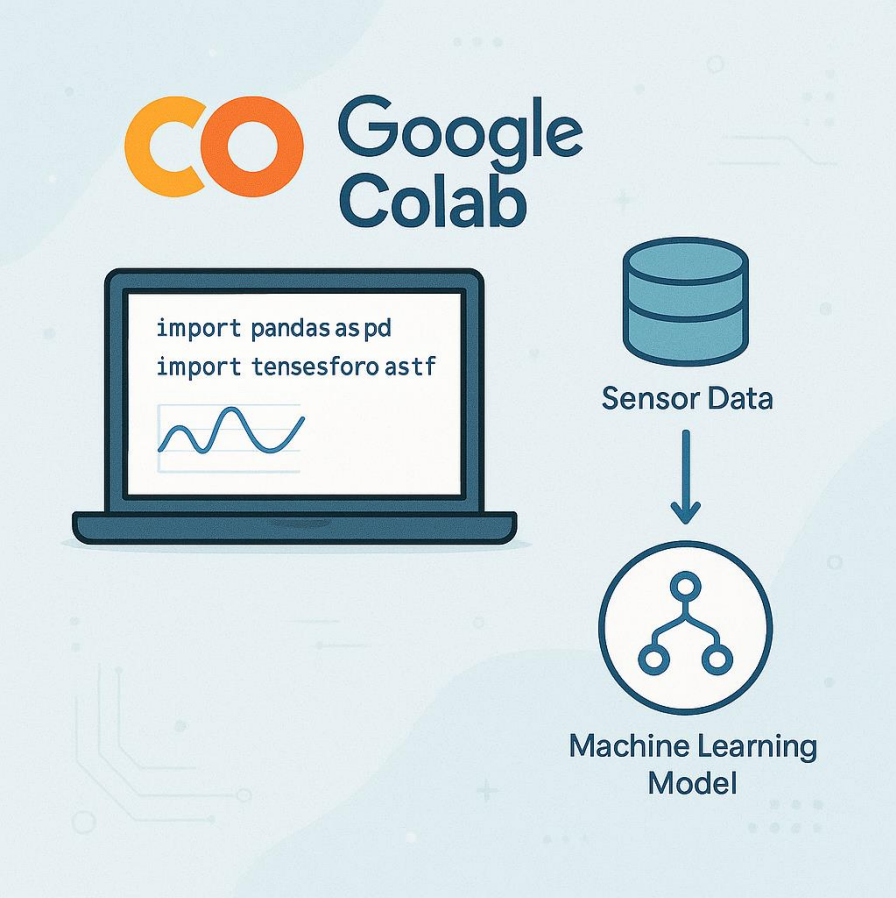
Python is a high-level, versatile programming language widely used in areas such as data science, artificial intelligence, web development, and automation. Known for its clean syntax and readability, Python allows developers to write efficient and maintainable code. In projects like the E-Tongue system for beverage analysis, Python plays a critical role in collecting and

processing sensor data, implementing machine learning algorithms (like Decision Trees), and visualizing results. With powerful libraries such as pandas, scikit-learn, and matplotlib, Python simplifies complex data analysis and model training processes, making it an ideal choice for real-time applications in quality control and smart monitoring systems.

THINGSPEAK:

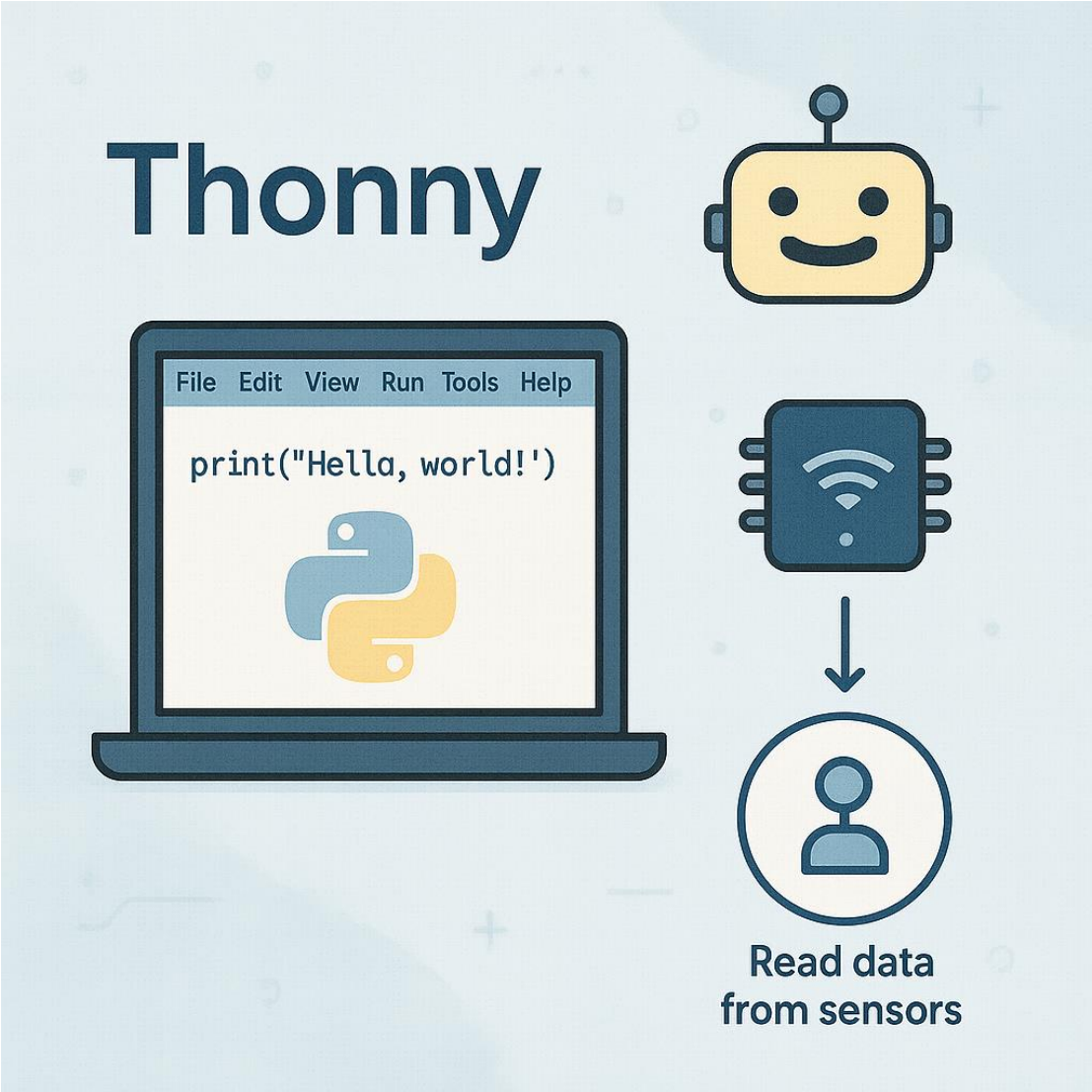
ThingSpeak is an open-source Internet of Things (IoT) platform that allows users to collect, store, analyze, and visualize real-time sensor data in the cloud. It provides a user-friendly web-based dashboard where sensor readings—such as pH, temperature, conductivity, and gas levels—can be continuously monitored. In the context of this project, ThingSpeak plays a crucial role in displaying the E-Tongue sensor data in real-time, enabling users to track beverage quality from anywhere. The platform supports integration with MATLAB for advanced analytics and allows triggers or alerts to be set based on threshold values. Its ability to manage multiple data streams through channels makes it a reliable and scalable solution for monitoring beverage safety and flavor profiles remotely.

GOOGLE COLAB:



Google Colaboratory, commonly known as **Google Colab**, is a free, cloud-based platform that allows users to write and execute Python code through a web browser. It is especially popular among data scientists, researchers, and students for developing and testing machine learning models without the need for powerful local hardware. Google Colab supports Jupyter notebooks and provides access to free GPUs and TPUs, making it ideal for training models efficiently. In the context of this project, Google Colab can be used to import sensor datasets, preprocess data, implement machine learning models (like Decision Trees), and visualize the results in a streamlined and collaborative environment. It also integrates well with Google Drive for easy storage and sharing of notebooks and datasets.

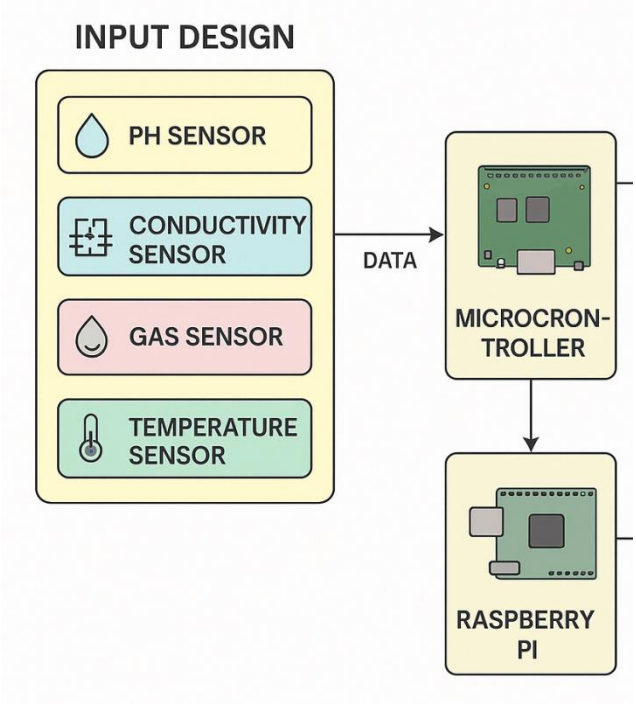
THONNY IDE:



Thonny is a beginner-friendly integrated development environment (IDE) designed specifically for learning and writing Python code. Developed by the University of Tartu in Estonia, Thonny offers a clean and simple interface that makes it ideal for students and hobbyists. It includes useful features such as a built-in debugger, variable explorer, and easy-to-understand error messages, which help users understand how Python programs execute step by step. Thonny is also compatible with microcontroller programming, such as **Raspberry Pi** or **ESP32**, making it a great tool for projects involving sensors and IoT. In this project, Thonny can

SYSTEM DESIGN

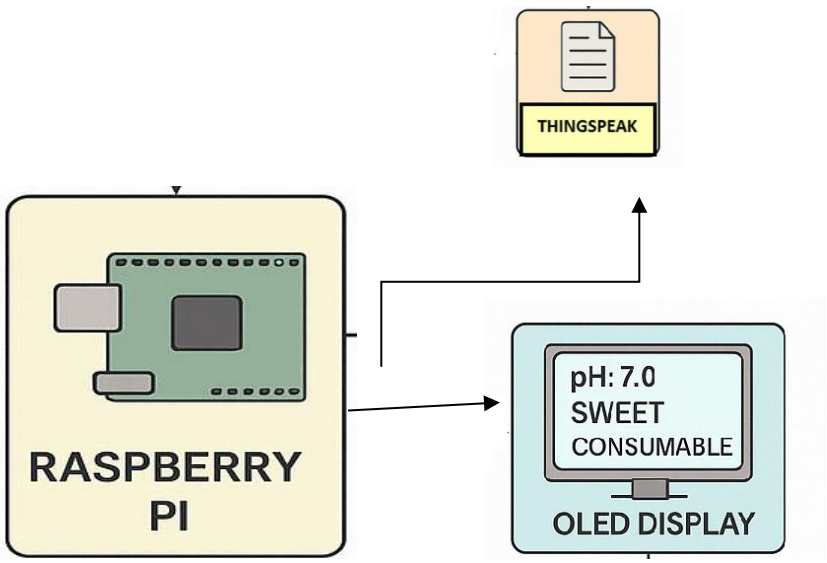
INPUT DESIGN



Component	Function
pH Sensor	Measures the acidity or alkalinity of the beverage.
Conductivity Sensor	Determines the sweetness (based on dissolved salts/sugars).
Gas Sensor (MQ-3)	Detects volatile compounds that may indicate spoilage.

Temperature Sensor	Checks the temperature of the beverage.
Humidity Sensor	Monitors surrounding environmental humidity.

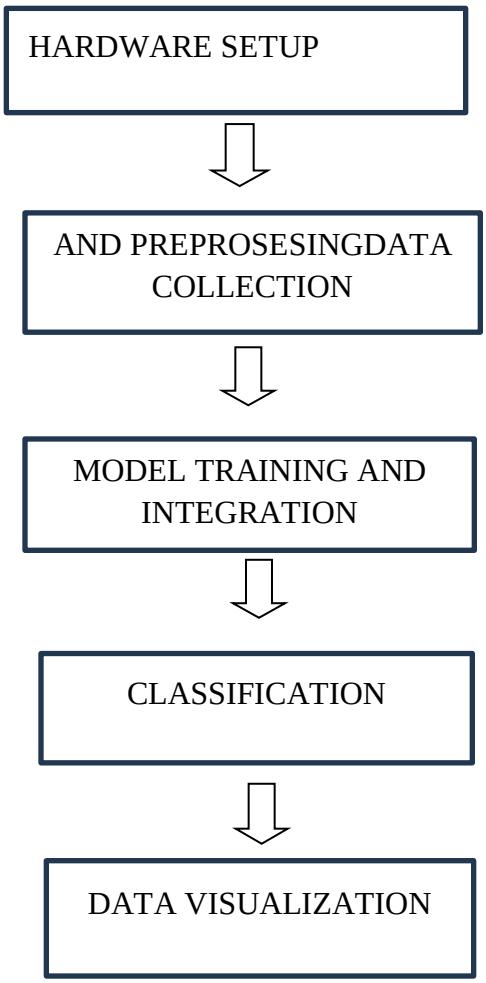
OUTPUT DESIGN



Component	Function
OLED Display	Displays real-time results: pH, taste type, gas levels, and final classification.
ThingSpeak (Cloud)	Remote data logging and monitoring of beverage stats.

SYSTEM DEVELOPMENT

METHODOLOGY



Module1 : Hardware setup

- **ARDUINO:**
  - Arduino is used to read values from sensor and sending them to raspberry pi.

- **SENSORS CONNECTED TO ARDUINO**:
  - GLASS PH sensor (to detect pH level)
  - Temperature DHT11 sensor(to detect the temperature)
  - GAS sensor (to detect gases present)
  - Conductivity sensor (to detect sweetness)

- **COLLECTION OF DATA**:
  - Arduino collect real-time data from sensors and send those data to raspberry pi through USB

- **RASPBERRY PI:**
  - It is used to get data from the Arduino using USB connection and used to send the dataset to thingspeak.
  - Model is trained with the collected data in the thonny IDE
  - The beverage is classified by the trained model(decision tree)to classify the beverage as consumable or not
  - The result is displayed in the OLED

- **GLASS pH ELECTRODE**:

- PH sensor is used to detect ph values consumable -3.0 to 8.5 not consumable below 3.0 and above 8.5.
- Sourness is generally associated with acidic substances having a pH below 7, while bitterness is often linked to alkaline or basic substances with a pH above 7

➢ Caliburation steps:

- Rinse the electrode-clean the electrode with distilled water.
- Caliburate with buffer-rinse the electrode in 4.00 and 7.00 each solution and collect the output.

➢ Formula for calculating pH:

$$pH = pHcal - \frac{(Vmeans - Vcal)}{S}$$

- Based on the pH level detected by the glass pH electrode the given beverage is verified for the overall consumption.

➢ CONDUCTIVITY SENSOR:

- A conductivity sensor measures how well a solution conducts electricity, which is useful for detecting sweetness (non-ionic compounds like sugar.

o Conductivity Calibration for Sweetness:

- **Prepare Calibration Solutions**: Use commercial calibration solutions (84 µS/cm, 1413 µS/cm, 12.88 mS/cm).

- **Clean the Sensor**: Rinse with distilled water and dry it.
- **Immerse the Sensor**: Place it in the first calibration solution.
- **Record the Reading**: Read and compare it with the expected value.
- **Apply Correction** Factor: If necessary, apply a correction factor in your code.
- **Repeat for Other Calibration Points**: Ensure multi-point calibration for a more accurate range.

➢ GAS SENSOR:

- A gas sensor can detect the air quality around a beverage by analyzing volatile organic compounds (VOCs), CO_2, and other gases released from the drink. This can be useful for assessing freshness, fermentation, contamination, or spoilage.

➢ TEMPERATURE AND HUMIDITY SENSOR:

- A temperature sensor helps monitor the storage conditions of beverages, ensuring optimal freshness, fermentation control, and preventing spoilage.

Module2: DATA COLLECTION

➢ The data collected using sensor:

In this module, data is collected from sensors by connecting them to Arduino.

- **pH Sensor:** Measures the acidity or alkalinity of the beverage.

- **Gas Sensor (MQ Series):** Detects volatile compounds and gases in the beverage.
- **Temperature Sensor (DHT11):** Measures the beverage temperature.
- **Conductivity Sensor :** Determines the salt and sugar concentration.

Module3: MODEL TRAINING AND INTEGRATION

➢ Model training:

- To assess beverage quality, a decision tree model is trained using sensor data (pH, gas concentration, temperature, conductivity). The training process involves:
- **Step 1:** Data Preparation
- **Step 2:** Training the decision tree model
- **Step 3:** Model Validation

➢ Model integration:

- The trained decision tree model for beverage quality assessment can be integrated with a Raspberry Pi to process real-time sensor data and classify beverage quality. The system will collect pH, gas, temperature, and conductivity readings from sensors connected to the Raspberry Pi, preprocess the data, and use the trained decision tree model to make quality predictions

MODULE4: CLASSIFICATION

- The decision tree model integrated with raspberry pi by consider the factors like pH, gas concentration, temperature, and conductivity impact quality and classified whether the beverage is consumable or not.

MODULE 5: Data visualization

- The thingspeak visualizes sensor data using line chart, bar graph and gauges and used to store and analyse data of beverage.

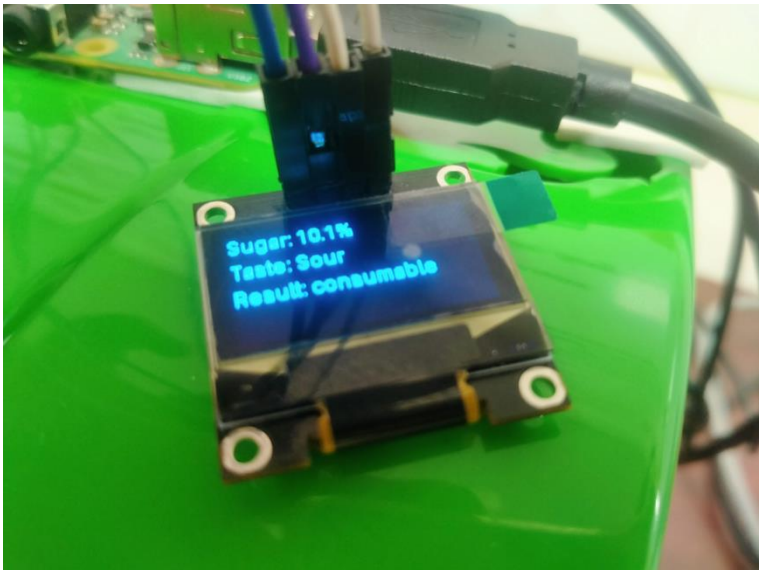
CONCLUSION

The development of the “E-Tongue for Beverage Flavor Profiling and Quality Analysis using AI” presents a significant advancement in the domain of food and beverage safety. By mimicking the human sense of taste through a combination of pH, conductivity, temperature, and gas sensors, the system offers a reliable and intelligent method to analyze the chemical composition and quality of beverages. The integration of machine learning, specifically the decision tree model, enhances the system’s accuracy in classifying beverages as either consumable or non-consumable.

This innovative approach not only ensures better quality control but also aligns with modern food safety standards by providing real-time monitoring and data-driven analysis. The system's ability to detect harmful substances and differentiate between various taste profiles contributes to improved consumer health and trust. Overall, the E-Tongue stands as a powerful tool in automating beverage testing, offering scalable, efficient, and intelligent quality assessment in the food industry.

SCREEN SHOTS

CONSUMABLE/ NOT CONSUMABLE




Sugar: 29.1%
Taste: Bitter
Result: Not consumable

SAMPLE CODE

MODEL CODE(DECISION TREE MODEL)

```
import joblib
import board
import busio
import adafruit_ssd1306
from PIL import Image, ImageDraw, ImageFont
import requests

# Load the model and scaler
model = joblib.load("/home/pi/etongue/dt_model.pkl")
scaler = joblib.load("/home/pi/etongue/scaler.pkl")

# ThingSpeak Configuration
THINGSPEAK_API_KEY = "R43`U"
THINGSPEAK_URL = "https://api.thingspeak.com/update"

# OLED setup
i2c = busio.I2C(board.SCL, board.SDA)
oled = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)
oled.fill(0)
oled.show()

# PIL image for drawing
image = Image.new("1", (oled.width, oled.height))
draw = ImageDraw.Draw(image)
font = ImageFont.load_default()

# Rule-based check
def is_consumable(temp, ph, gas, sugar):
    return (
        0 <= temp <= 50 and
        3.0 <= ph <= 7.5 and
        gas <= 100 and
```

```
        0 <= sugar <= 15
    )

# Taste classification
def taste_type(ph):
    return "Sour" if ph < 7 else "Bitter"
import joblib
import board
import busio
import adafruit_ssd1306
from PIL import Image, ImageDraw, ImageFont
import requests

# Load the model and scaler
model = joblib.load("/home/pi/etongue/dt_model.pkl")
scaler = joblib.load("/home/pi/etongue/scaler.pkl")

# ThingSpeak Configuration
THINGSPEAK_API_KEY = "R43`U"
THINGSPEAK_URL = "https://api.thingspeak.com/update"

# OLED setup
i2c = busio.I2C(board.SCL, board.SDA)
oled = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)
oled.fill(0)
oled.show()

# PIL image for drawing
image = Image.new("1", (oled.width, oled.height))
draw = ImageDraw.Draw(image)
font = ImageFont.load_default()

# Rule-based check
def is_consumable(temp, ph, gas, sugar):
    return (
#ph = float(input("pH (e.g., 4.5): "))
sugar = float(input("Sugar (g/L): "))
temp = float(input("Temperature (°C): "))
gas = float(input("Gas (ppm): ")))
```

```
# Rule-based + model prediction
if is_consumable(temp, ph, gas, sugar):
    prediction = 1
    status = "consumable"
else:
    sample = [[ph, sugar, temp, gas]]
    scaled = scaler.transform(sample)
    prediction = model.predict(scaled)[0]
    status = "consumable" if prediction == 1 else "Not consumable"

# Taste type
taste = taste_type(ph)

# Display on OLED
draw.rectangle((0, 0, oled.width, oled.height), outline=0, fill=0)
draw.text((0, 0), f"Sugar: {sugar}%", font=font, fill=255)
draw.text((0, 16), f"Taste: {taste}", font=font, fill=255)
draw.text((0, 32), f"Result: {status}", font=font, fill=255)
oled.image(image)
oled.show()

# Print to console
print(f"Sugar: {sugar}%")
print(f"Taste: {taste}")
print(f"Prediction: {status}")

# Send to ThingSpeak
payload = {
    "api_key": THINGSPEAK_API_KEY,
    "field1": temp,
    "field2": ph,
    "field3": sugar,
    "field4": gas,
    "field5": 1 if status == "consumable" else 0,
    "field6": 1 if taste == "Sour" else 2
}

response = requests.get(THINGSPEAK_URL, params=payload)
if response.status_code == 200:
    print("Data sent to ThingSpeak successfully!")
```

```
else:
    print("Failed to send data to ThingSpeak:", response.text)
```

LAST CODE:

```
import serial
import joblib
import board
import busio
import adafruit_ssd1306
from PIL import Image, ImageDraw, ImageFont
import requests
import time

# Load ML model and scaler
model = joblib.load("/home/pi/etongue/dt_model.pkl")
scaler = joblib.load("/home/pi/etongue/scaler.pkl")

# ThingSpeak Configuration
THINGSPEAK_API_KEY = "R43IVOU7MY0R8Q95"  # Replace with actual key
THINGSPEAK_URL = "https://api.thingspeak.com/update"

# OLED setup
i2c = busio.I2C(board.SCL, board.SDA)
oled = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)
oled.fill(0)
oled.show()

# PIL image for drawing
image = Image.new("1", (oled.width, oled.height))
draw = ImageDraw.Draw(image)
font = ImageFont.load_default()

# Rule-based check
def is_consumable(temp, ph, gas, sugar):
    return (
        0 <= temp <= 50 and
        3.0 <= ph <= 7.5 and
        gas <= 120 and
```

```
        0 <= sugar <= 15
    )

# Taste classification
def taste_type(ph):
    return "sour" if ph < 7 else "bitter"

# Read from Arduino Serial
ser = serial.Serial('/dev/ttyACM0', 115200, timeout=2)  # Adjust port if needed
time.sleep(2)  # Wait for serial to initialize

while True:
    try:
        line = ser.readline().decode().strip()
        if not line:
            continue

        # Expecting: temp,ph,cond,sugar,gas
        values = line.split(",")
        if len(values) != 5:
            print("Invalid data format:", line)
            continue

        temp = float(values[0])
        ph = float(values[1])
        cond = float(values[2])
        sugar = float(values[3])
        gas = float(values[4])

        # Rule + Model
        if is_consumable(temp, ph, gas, sugar):
            prediction = 1
            status = "consumable"
        else:
            sample = [[ph, sugar, temp, gas]]
            scaled = scaler.transform(sample)
            prediction = model.predict(scaled)[0]
            status = "consumable" if prediction == 1 else "Not consumable"

        taste = taste_type(ph)
```

```
    # Display on OLED
    draw.rectangle((0, 0, oled.width, oled.height), outline=0, fill=0)
    draw.text((0, 0), f"Sugar: {sugar:.1f}%", font=font, fill=255)
    draw.text((0, 16), f"Taste: {taste}", font=font, fill=255)
    draw.text((0, 32), f"Result: {status}", font=font, fill=255)
    oled.image(image)
    oled.show()

    # Console output
    print(f"Temperature: {temp}°C")
    print(f"pH: {ph}")
    print(f"Sugar: {sugar}%")
    print(f"Gas: {gas} ppm")
    print(f"Taste: {taste}")
    print(f"Prediction: {status}")

    # Send to ThingSpeak
    payload = {
        "api_key": THINGSPEAK_API_KEY,
        "field1": temp,
        "field2": ph,
        "field3": sugar,
        "field4": gas,

    }

    response = requests.get(THINGSPEAK_URL, params=payload)
    if response.status_code == 200:
        print("Data sent to ThingSpeak!")
    else:
        print("ThingSpeak Error:", response.text)

    time.sleep(2)  # Match Arduino interval

except Exception as e:
    print("Error:", e)
    continue
```